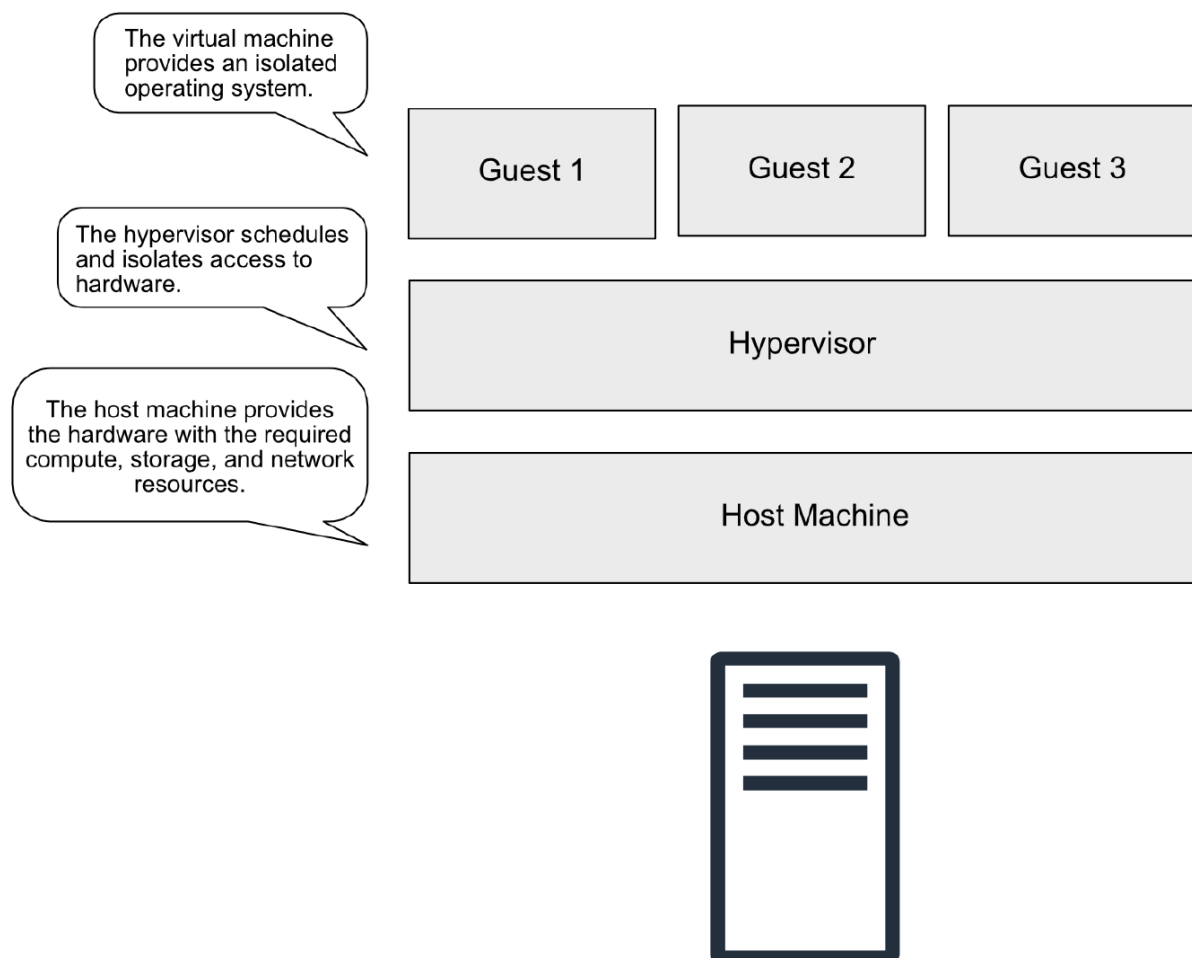


Exploring a virtual machine

A virtual machine (VM) runs on a physical machine isolated from other virtual machines by the hypervisor; it consists of CPUs, memory, networking interfaces, and storage. The physical machine is called the host machine, and the VMs running on it are called guests. A hypervisor is responsible for isolating the guests from each other and for scheduling requests to the hardware, by providing a virtual hardware platform to the guest system. Figure shows these layers of virtualization.



Launching a virtual machine

It takes only a few clicks to launch a virtual machine, which AWS calls an EC2 instance:

1. Open the AWS Management Console at console.aws.amazon.com.
2. Make sure you're in the N. Virginia (US East) region (see figure), because we optimized our examples for this region.
3. Click on Services and search for EC2.

4. Click Launch instance to start the wizard for launching a virtual machine as shown in Figure.

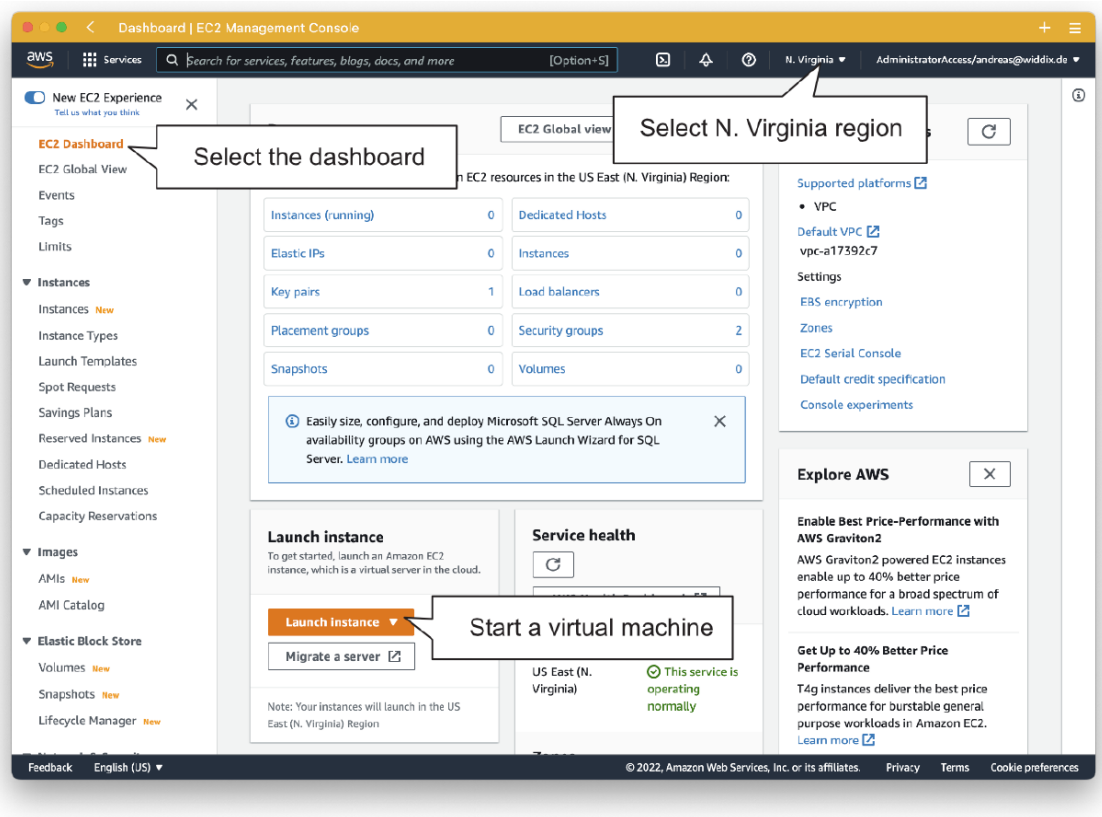


Figure 3.2 The EC2 Dashboard gives you an overview of all parts of the service.

A form will show up, guiding you through all the details needed to create a virtual machine:

1. Naming the virtual machine
2. Selecting the operating system (OS)
3. Choosing the size of your virtual machine
4. Configuring key pair
5. Configuring a Network and firewall settings
6. Adding storage

NAMING THE VIRTUAL MACHINE

To make it easy to find your virtual machine later, it is recommended to assign a name to it. That's especially important, when other people have access to the same AWS account. Figure shows the details.

Name and tags

Name

mymachine

Type in a name for the virtual machine

[Add additional tags](#)

Figure 3.3 Naming the virtual machine

SELECTING THE OPERATING SYSTEM

Next, you need to choose an operating system (OS). In AWS, the OS comes bundled with preinstalled software for your virtual machine; this bundle is called an Amazon Machine Image (AMI). Select Amazon Linux 2 AMI (HVM), as shown in figure

Application and OS Images (Amazon Machine Image) [Info](#)

An AMI is a template that contains the software configuration (operating system, application server, and applications) required to launch your instance. Search or Browse for AMIs if you don't see what you are looking for below

Search our full catalog including 1000s of application and OS images

Recents Quick Start

Amazon Linux Ubuntu Windows Red Hat SUSE Linux

Select Amazon Linux

Amazon Machine Image (AMI)

Amazon Linux 2 AMI (HVM) - Kernel 5.10, SSD Volume Type
ami-0c02f055956c7d316 (64-bit (x86)) / ami-03190fe20ef6b1419 (64-bit (Arm))
Virtualization: hvm ENA enabled: true Root device type: ebs

Choose Amazon Linux 2

Description

Amazon Linux 2 Kernel 5.10 AMI 2.0.20220316.0 x86_64 HVM gp2

Architecture AMI ID

64-bit (x86)

Use 64-bit x86 architecture

Figure 3.4 Choose the OS for your virtual machine.

- AMI stands for Amazon Machine Image.
- AMI provides the information required to launch the EC2 instance. AMI includes the operating system, application server, and applications needed to launch an EC2 instance.

- AMI includes the pre-configured templates of the operating system that runs on the AWS
- It may also contain additional software, libraries, and configuration settings required for specific use cases.
- Users can launch multiple instances with the same configuration from a single AMI
- It is a template used to create virtual machines, known as Amazon Elastic Compute Cloud (EC2) instances, within the Amazon Web Services (AWS) environment.

CHOOSING THE SIZE OF YOUR VIRTUAL MACHINE

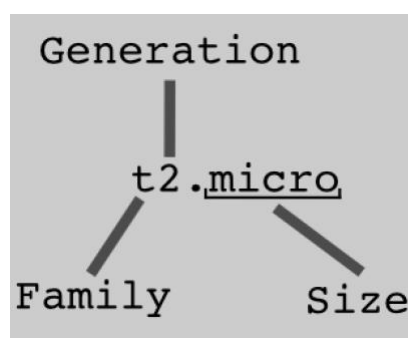
It's now time to choose the computing power needed for your virtual machine as shown in figure. AWS classifies computing power into instance types. An instance type primarily describes the number of virtual CPUs and the amount of memory.

Instance types and families

The names for different instance types are all structured in the same way. The instance family groups instance types with similar characteristics. AWS releases new instance types and families from time to time; the different versions are called generations. The instance size defines the capacity of CPU, memory, storage, and networking.

The instance type t2.micro tells you the following:

- The instance family is called t. It groups small, cheap virtual machines with low baseline CPU performance but the ability to burst significantly over baseline CPU performance for a short time.
- You're using generation 2 of this instance family.
- The size is micro, indicating that the EC2 instance is very small.



There are instance families optimized for different kinds of use cases.

T family—Cheap, moderate baseline performance with the ability to burst to higher performance for short periods of time

M family—General purpose, with a balanced ration of CPU and memory

C family—Computing optimized, high CPU performance

R family—Memory optimized, with more memory than CPU power compared to M family

X family—Extensive capacity with a focus on memory, up to 1952 GB memory and 128 virtual cores

D family—Storage optimized, offering huge HDD capacity

I family—Storage optimized, offering huge SSD capacity

P, G, and CG family—Accelerated computing based on GPUs (graphics processing units)

F family—Accelerated computing based on FPGAs (field programmable gate arrays)

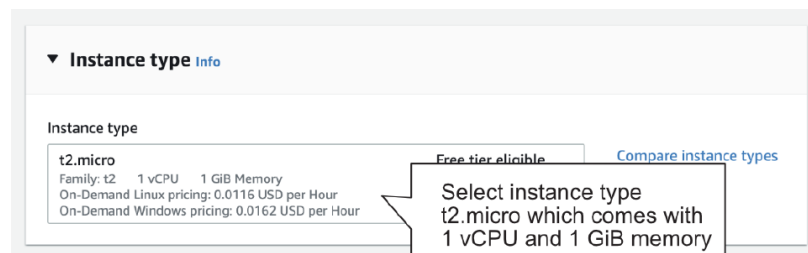


Figure 3.7 Choosing the size of your virtual machine

CONFIGURING THE KEY PAIR FOR LOGIN

As an administrator of a Linux machine, you used username and password or username and a public/private key pair to authenticate yourself in the past. By default, AWS uses username and a key pair for authentication.

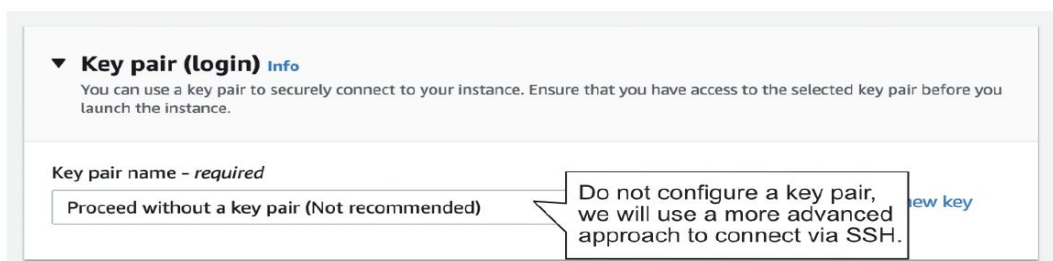
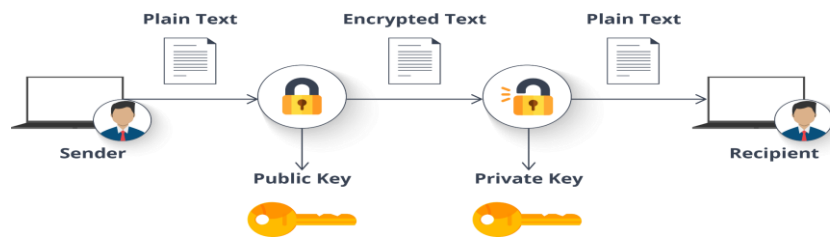


Figure 3.8 Proceed without a key pair

- **Public key + Private Key = Key Pair**
- Amazon EC2 uses public and private key cryptography to encrypt and decrypt information while connecting to EC2 instance
- Public key is used to encrypt the data, while the recipient uses a private key to decrypt the same
- AWS provides a **.pem** file, and the client needs to convert it to **.ppk** format which is recognized by client software



A key is required to provide authentication to an instance.

DEFINING NETWORK AND FIREWALL SETTINGS

In the next section it is up to you, to configure the network and firewall settings for the EC2 instance. The defaults are fine for now. With the configuration shown in figure the firewall does not allow any incoming connections at all.

▼ Network settings

Info

Edit

Network

Info

vpc-a17392c7

Subnet

Info

No preference (Default subnet in any availability zone)

Auto-assign public IP

Info

Enable

Firewall (security groups)

Info

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

Create security group

Select existing security group

We'll create a new security group called 'launch-wizard-1' with the following rules:

Allow SSH traffic from

Helps you connect to your instance

Allow HTTPs traffic from the int

To set up an endpoint, for example when creating a web server

Allow HTTP traffic from the internet

To set up an endpoint, for example when creating a web server

Keep the default network.

Ensure assigning a public IP is enabled

Creates a new firewall configuration named launch-wizard-1

Deselect inbound SSH traffic, as we will use a more advanced approach to connect to the VM.

- A security group, like a virtual firewall, controls the traffic for one or more instances
- The traffic can either be *inbound or outbound* from an instance
- It secures the instance using *IP protocol, port access*, and through the *source or destination address (rules)*
- Instances associated with the security group cannot talk to each other if there is no *explicit rule (IAM)* to allow it

ATTACHING STORAGE

Next, attach some storage to your virtual machine used for the root file system. It is fine to keep the defaults and attach a volume with 8 GB of type gp2 which consists of network-attached SSDs as illustrated in figure.

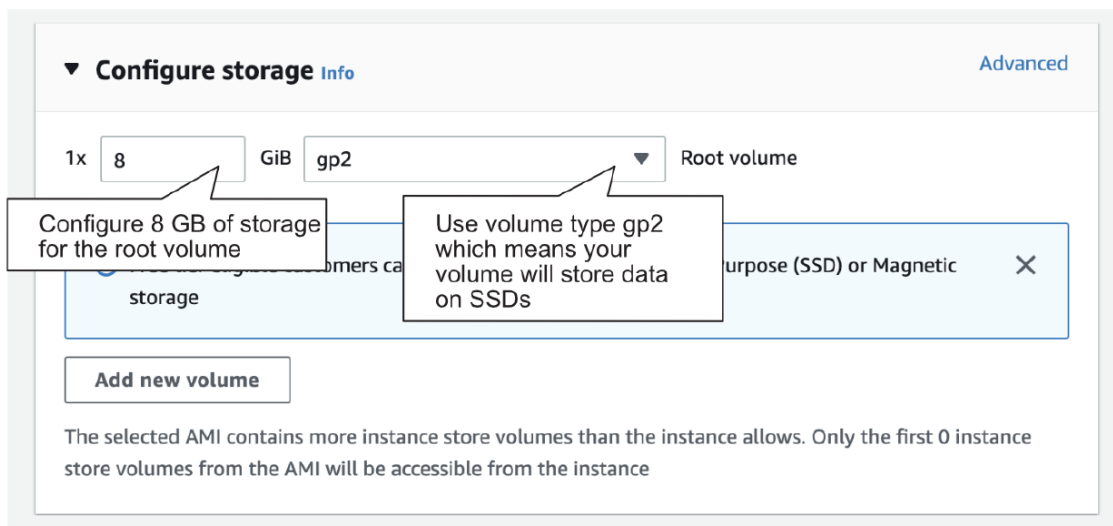


Figure 3.10 Adding storage to the virtual machine

LAUNCHING THE EC2 INSTANCE

You are ready to launch your virtual machine. To do so, just press the *Launch instance* button as illustrated in figure.

▼ **Summary**

Number of instances [Info](#)
1

Software Image (AMI)
Amazon Linux 2 Kernel 5.10 AMI...[read more](#)
ami-0c02fb55956c7d316

Virtual server type (instance type)
t2.micro

Firewall (security group)
New security group

Storage (volumes)
1 volume(s) - 8 GiB

Free tier: In your first year includes 750 hours of t2.micro (or t3.micro in the Regions in which t2.micro is unavailable) instance usage on free tier AMIs per month, 30 GiB of EBS storage, 2 million IOs, 1 GB of snapshots, and 100 GB of bandwidth to the internet

Cancel **Launch instance**

Figure 3.15 Launching the EC2 instance

Connecting to your virtual machine

The following instructions guide you through the steps necessary to connect to the EC2 instance you launched in the previous section which fulfills all these requirements already:

1. Choose Instances from the sub navigation of the EC2 service, in case you are not looking at the list of EC2 instances already as shown in figure.
2. Select the instance named mymachine.
3. Wait until the instance reaches status Running.
4. Click the Connect button.

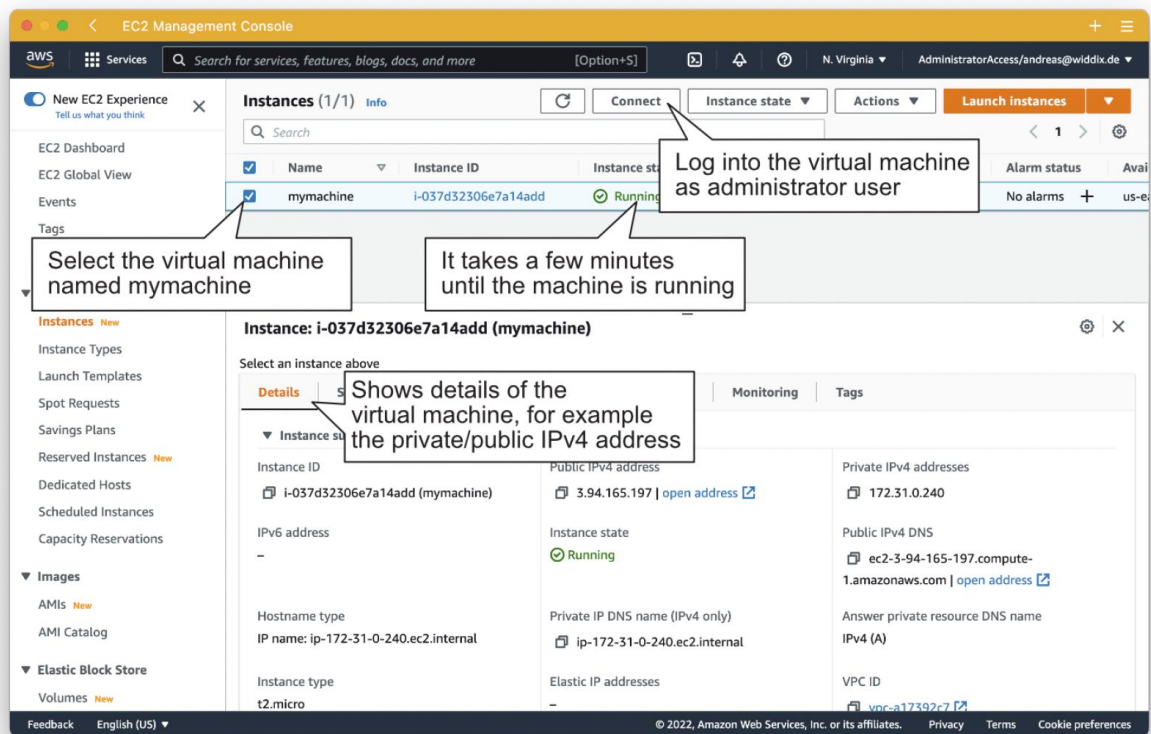


Figure 3.16 Listing all EC2 instances running in the region N. Virginia

5. Select the Session Manager tab.
6. Press the Connect button as shown in figure

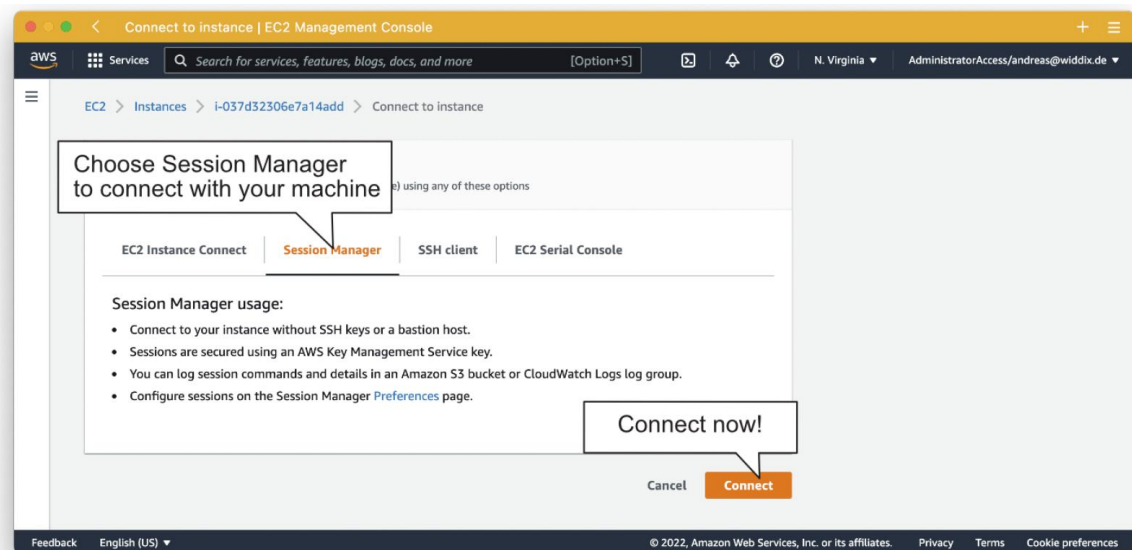


Figure 3.17 Connecting to an EC2 instance via Session Manager

After a few seconds, a terminal appears in your browser window as shown in figure

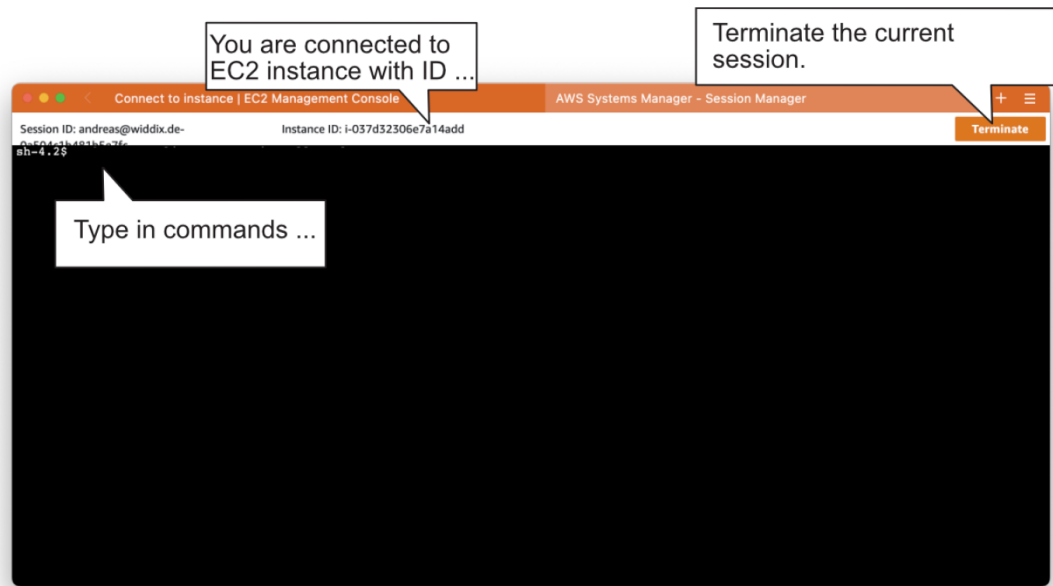


Figure 3.18 The terminal connected with your EC2 instance is waiting for input

Monitoring, debugging, shutting down, and changing the size of a virtual machine

Monitoring and debugging a virtual machine

If you need to find the reason for an error or why your application isn't behaving as you expect, it's important to have access to tools that can help with monitoring and debugging. AWS provides tools that let you monitor and debug your virtual machines. One approach is to examine the virtual machine's logs.

Showing logs from a virtual machine

If you need to find out what your virtual machine was doing during and after startup, there is a simple solution. AWS allows you to see the EC2 instance's logs with the help of the Management Console (the web interface you use to start and stop virtual machines). Follow these steps to open your VM's logs:

1. Go to EC2 in the AWS Management Console: console.aws.amazon.com/ec2/.
2. Open the list of all virtual machines choosing Instances from the sub navigation.
3. Select the running virtual machine by clicking the row in the table.
4. In the Actions menu, choose Monitor and troubleshoot and Get system log.
5. A screen showing the system logs from your VM that would normally be displayed on a physical monitor during startup as shown in figure appears.

The log contains all log messages that would be displayed on the monitor of your machine if you were running it on-premises. Watch out for any log messages stating that an error occurred during startup. If the error message is not obvious, you should contact the vendor of the AMI, AWS Support, or post your question in the AWS Developer Forums at forums.aws.amazon.com.

This is a simple and efficient way to access your system logs without needing an SSH connection. Note that it will take several minutes for a log message to appear in the log viewer.

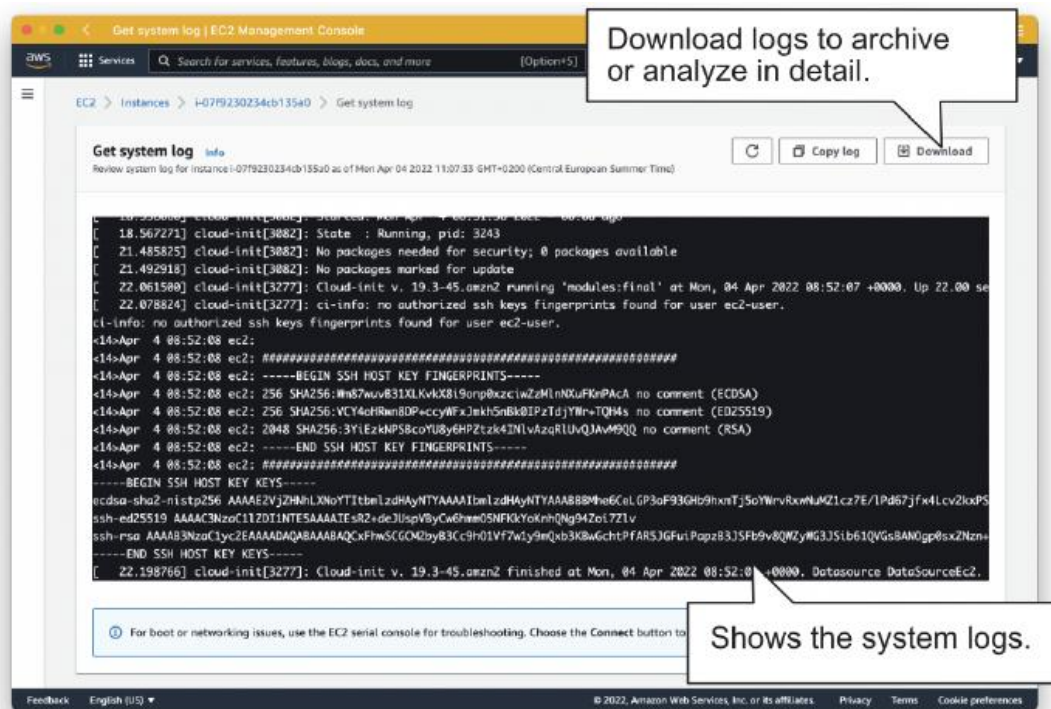


Figure 3.19 Debugging a virtual machine with the help of system logs

Monitoring the load of a virtual machine

AWS can help you answer another question: is your virtual machine close to its maximum capacity? Follow these steps to open the EC2 instance's metrics:

1. Go to EC2 in the AWS Management Console: console.aws.amazon.com/ec2/.
2. Open the list of all virtual machines choosing Instances from the sub navigation.
3. Select the running virtual machine by clicking the row in the table.
4. Select the Monitoring tab at lower right.
5. Click the three dots at the upper-right of the Network in (bytes) metric and choose Enlarge.

You'll see a graph that shows the virtual machine's utilization of incoming networking traffic, similar to figure Figure 3.20. There are metrics for CPU, network, and disk usage. As AWS is looking at your VM from the outside, there is no metric indicating the memory usage. You can publish a memory metric yourself, if needed. The metrics are updated every 5 minutes if you use basic monitoring, or every minute if you enable detailed monitoring of your virtual machine, which costs extra.

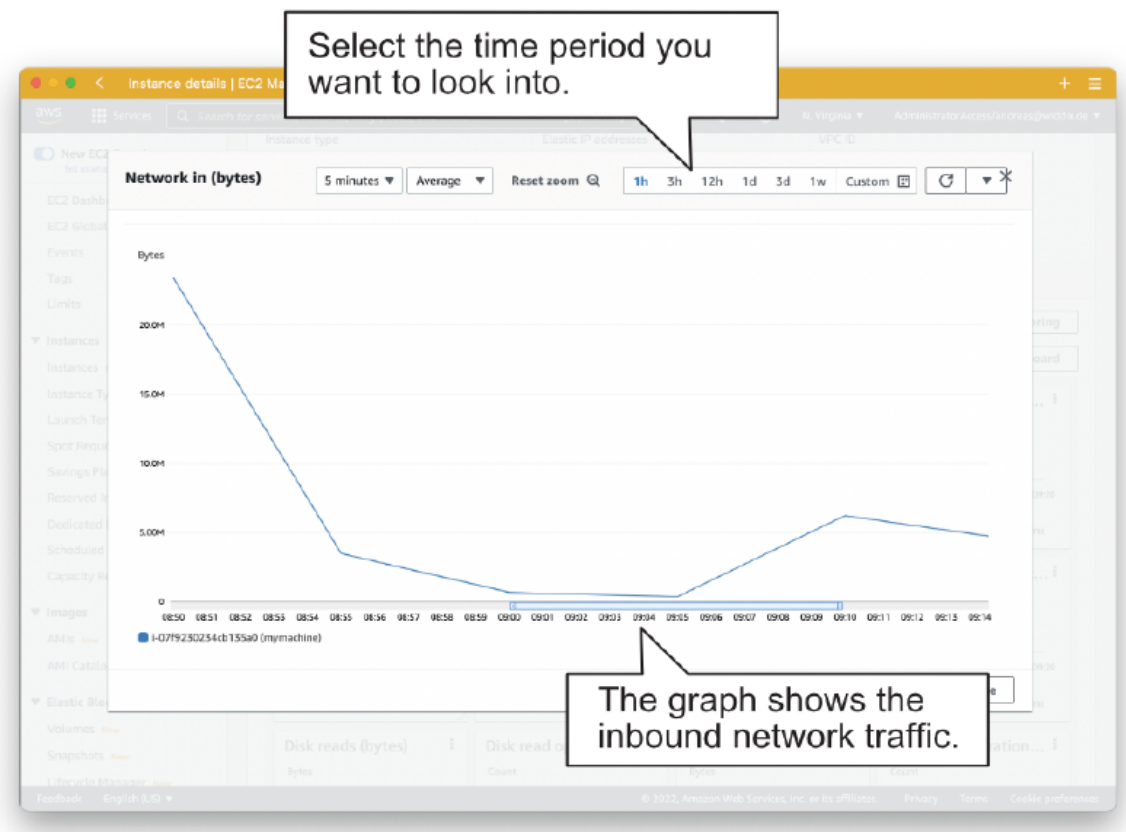


Figure 3.20 Gaining insight into a virtual machine's incoming network traffic with the CloudWatch metric

Checking the metrics of your EC2 instance is helpful when debugging performance issues.

Shutting down a virtual machine

To avoid incurring charges, you should always turn off virtual machines you're not using them. You can use the following four actions to control a virtual machine's state:

Start—You can always start a stopped virtual machine. If you want to create a completely new machine, you'll need to launch a virtual machine.

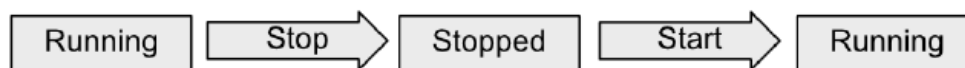
Stop—You can always stop a running virtual machine. A stopped virtual machine doesn't incur charges, except for attached resources like network-attached storage. A stopped virtual machine can be started again but likely on a different host. If you're using network-attached storage, your data persists.

Reboot—Have you tried turning it off and on again? If you need to reboot your virtual machine, this action is what you want. You won't lose any persistent data when rebooting a virtual machine because it stays on the same host.

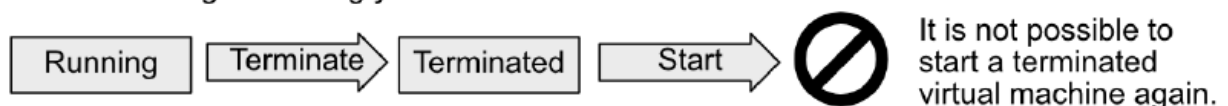
Terminate—Terminating a virtual machine means deleting it. You can't start a virtual machine that you've already terminated. The virtual machine is deleted, usually together with dependencies like network-attached storage and public and private IP addresses. A terminated virtual machine doesn't incur charges.

Figure illustrates the difference between stopping and terminating an EC2 instance, with the help of a flowchart.

It is always possible to stop a running machine and to start a stopped machine.



But terminating is deleting your virtual machine.



Stopping or terminating unused virtual machines saves costs and prevents you from being surprised by an unexpected bill from AWS. You may want to stop or terminate unused virtual machines when:

- You have launched virtual machines to implement a proof-of-concept. After finishing the project, the virtual machines are no longer needed. Therefore, you can terminate them.
- You are using a virtual machine to test a web application. As no one else uses the virtual machine, you can stop it before you knock off work, and start it back up again the following day.
- One of your customers canceled their contract. After backing up relevant data, you can terminate the virtual machines that had been used for your former customer.

After you terminate a virtual machine, it's no longer available and eventually disappears from the list of virtual machines.

Changing the size of a virtual machine

It is always possible to change the size of a virtual machine. This is one of the advantages of using the cloud, and it gives you the ability to scale vertically. If you need more computing power, increase the size of the EC2 instance or vice versa.

In this section, you'll learn how to change the size of a running virtual machine. To begin, follow these steps to start a small virtual machine:

1. Go to EC2 in the AWS Management Console: console.aws.amazon.com/ec2/.
2. Click the Launch instances button.
3. Type in growingup as the name of the virtual machine.
4. Choose the Amazon Linux 2 AMI.
5. Select instance type t2.micro.
6. Select the option Proceed without a key pair.
7. Keep the default for network and storage.
8. Select the IAM instance profile ec2-ssm-core under advanced details.
9. Launch the instance.

You've now started an EC2 instance of type t2.micro. This is one of the smallest virtual machines available on AWS.

Use the Session Manager to connect to the instance as demonstrated in the previous section, and execute `cat /proc/cpuinfo` and `free -m` to see information about the machine's capabilities.

The output should look similar to this:

```
$ cat /proc/cpuinfo
processor       : 0
vendor_id      : GenuineIntel
cpu family     : 6
model          : 63
model name     : Intel(R) Xeon(R) CPU E5-2676 v3 @ 2.40GHz
stepping       : 2
microcode      : 0x46
cpu MHz        : 2399.915
cache size     : 30720 KB
[...]
```

```
$ free -m
              total        used        free      shared    buff/cache   available
Mem:           965         93        379          0         492         739
Swap:           0           0           0
```

Your virtual machine provides a single CPU core and 965 MB of memory. If your application is having performance issues, increasing the instance size can solve the problem.

If you need more CPUs, more memory, or more networking capacity, there are many other sizes to choose from. You can even change the virtual machine's instance family and generation. To increase the size of your VM, you first need to stop it:

1. Go to EC2 in the AWS Management Console: console.aws.amazon.com/ec2/.
2. Click Instances in the sub menu to jump to an overview of your virtual machines.
3. Select your running VM from the list by clicking it.
4. Click Instance state and Stop instance.

After waiting for the virtual machine to stop, you can change the instance type:

1. Click the Actions button and select Instance settings.
2. Click Change instance type.
3. Select m5.large as the new instance type and press the Apply button as shown in figure.

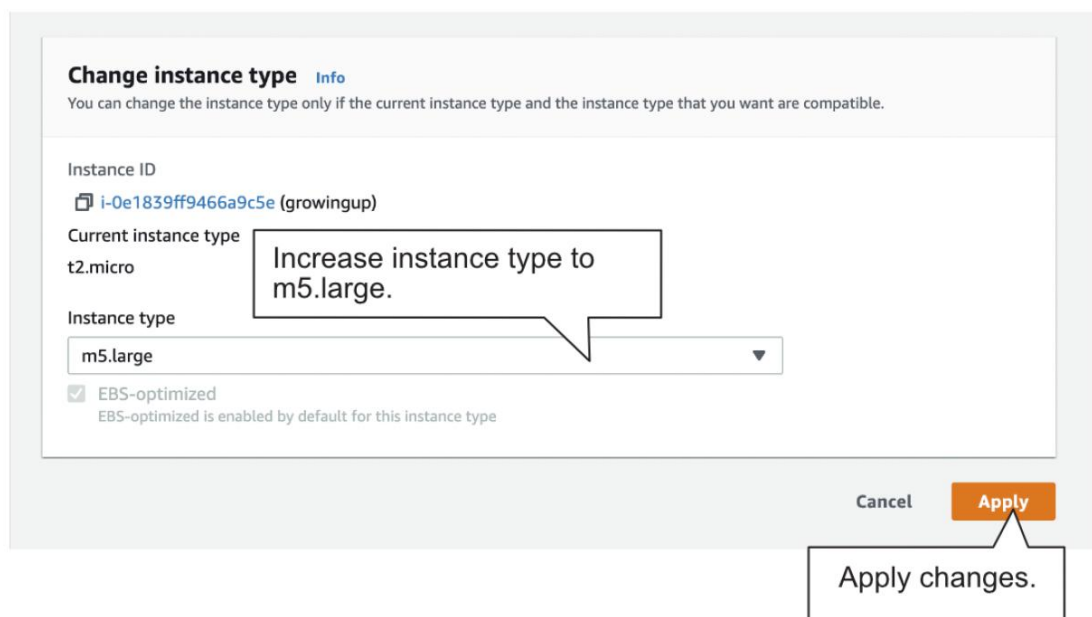


Figure 3.22 Increasing the size of your virtual machine by selecting m4.large as Instance Type

You've now changed the size of your virtual machine and are ready to start it again. To do so, select your EC2 instance and choose Start instance under Instance state. Your VM will start with more CPUs, more memory, and increased networking capabilities.

Use the Session Manager to connect to your EC2 instance, and execute `cat /proc/cpuinfo` and `free -m` to see information about its CPU and memory again. The output should look similar to this:


```
$ cat /proc/cpuinfo
processor       : 0
vendor_id     : GenuineIntel
cpu family    : 6
model         : 85
model name    : Intel(R) Xeon(R) Platinum 8259CL CPU @ 2.50GHz
stepping      : 7
microcode     : 0x500320a
cpu MHz       : 3117.531
cache size    : 36608 KB
[...]

processor       : 1
vendor_id     : GenuineIntel
cpu family    : 6
model         : 85
model name    : Intel(R) Xeon(R) Platinum 8259CL CPU @ 2.50GHz
stepping      : 7
microcode     : 0x500320a
cpu MHz       : 3100.884
cache size    : 36608 KB
[...]
```

```
$ free -m
              total        used        free      shared    buff/cache   available
Mem:          7737          108        7427           0          202        7406
Swap:           0           0           0
```

Your virtual machine can use two CPU cores and offers 7,737 MB of memory, compared to a single CPU core and 965 MB of memory before you increased the VM's size.

Allocating a public IP address, Adding an additional network interface to a virtual machine

Allocating a public IP address

Each VM was connected to a public IP address automatically. But every time you launched or stopped a VM, the public IP address changed. If you want to host an application under a fixed IP address, this won't work. AWS offers a service called Elastic IPs for allocating fixed public IP addresses.

Using a fixed public IP address is useful, in case clients aren't able to resolve a DNS name, a firewall rule based on IP addresses is required, or in case you do not want to update DNS records to avoid the delay until all clients resolve to the new IP address.

Therefore, allocate a public IP address and associate with your EC2 instance named syndey:

1. Open the Management Console, and go to the EC2 service.
2. Choose Elastic IPs from the submenu. You'll see an overview of public IP addresses allocated by you.
3. Allocate a public IP address by clicking Allocate Elastic IP address as shown in figure.

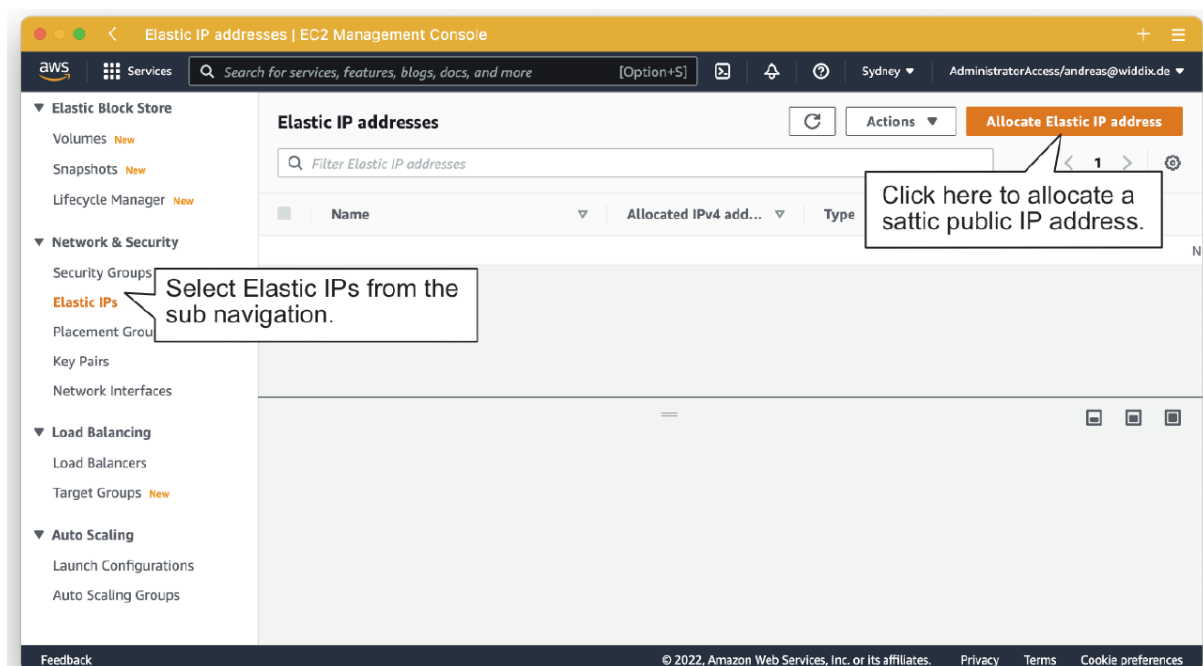


Figure 3.24 Overview of public IP addresses connected to your account in the current region

4. Select Amazon's pool of IPv4 addresses and click the Allocate button as shown in figure

Allocate Elastic IP address [Info](#)

Elastic IP address settings [Info](#)

Public IPv4 address pool

- ☒ Amazon's pool of IPv4 addresses
- ☐ Public IPv4 address that you bring to your own pool (no pools found) [Learn more](#)
- ☐ Customer owned pool of IPv4 addresses (option disabled because no customer owned pools found) [Learn more](#)

Global static IP addresses

AWS Global Accelerator can provide global static IP addresses that are announced worldwide using anycast from AWS edge locations. This can help improve the availability and latency for your user traffic by using the Amazon global network. [Learn more](#)

[Create accelerator](#)

Tags - optional

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

No tags associated with the resource.

[Add new tag](#)

You can add up to 50 more tag

[Cancel](#) [Allocate](#)

Figure 3.25 Allocating a public IPv4 address

- To associate the Elastic IP with your EC2 instance, select the public IP address you just allocated and click the Actions button and choose Associate Elastic IP address as demonstrated in figure.
- Select resource type Instance and select your EC2 instance named sydney from the dropdown list.
- Press the Associate button.

Allocating a static public IP address can be useful if you want to make sure the endpoint to your application doesn't change, even if you have to replace the virtual machine behind the scenes.

For example, assume that virtual machine A is running and has an associated Elastic IP. The following steps let you replace the virtual machine with a new one without changing the public IP address:

1. Start a new virtual machine B to replace running virtual machine A.
2. Install and start applications as well as all dependencies on virtual machine B.
3. Disassociate the Elastic IP from virtual machine A, and associate it with virtual machine B.

Requests using the Elastic IP address will now be routed to virtual machine B, with a short interruption while moving the Elastic IP. You can also connect multiple public IP addresses with a virtual machine by using multiple network interfaces, as described in the next section. This can be useful if you need to host different applications running on the same port, or if you want to use a unique fixed public IP address for different websites.

The screenshot shows the 'Associate Elastic IP address' page in the AWS Management Console. The page title is 'Associate Elastic IP address' and the subtitle is 'Choose the instance or network interface to associate to this Elastic IP address (13.236.254.54)'. The 'Elastic IP address' is 13.236.254.54. Under 'Resource type', 'Instance' is selected. A callout box points to this selection with the text 'Chose Instance as resource type.' Below this, a warning message states: 'If you associate an Elastic IP address to an instance that already has an Elastic IP address associated, this previously associated Elastic IP address will be disassociated but still allocated to your account. [Learn more](#)'. The 'Instance' dropdown menu is open, showing a list of instances with 'i-0c5b91f6d61f2d187' selected. A callout box points to this selection with the text 'Select the instance name sydney from the dropdown list.' The 'Private IP address' field is empty, with a placeholder 'Choose a private IP address'. The 'Reassociation' section has a checkbox 'Allow this Elastic IP address to be reassociated' which is unchecked. At the bottom right, there are 'Cancel' and 'Associate' buttons. A callout box points to the 'Associate' button with the text 'Associate the public IP address with your instance.'

Figure 3.26 Associating an Elastic IP address with an EC2 instance

Adding an additional network interface to a virtual machine

In addition to managing public IP addresses, you can control your virtual machine's network interfaces. It is possible to add multiple network interfaces to a VM and control the private and public IP addresses associated with those network interfaces.

Here are some typical use cases for EC2 instances with multiple network interfaces:

- Your web server needs to answer requests by using multiple TLS/SSL certificates, and you can't use the Server Name Indication (SNI) extension due to legacy clients.

- You want to create a management network separated from the application network, and therefore your EC2 instance needs to be accessible from two networks. Figure illustrates an example.
- Your application requires or recommends the use of multiple network interfaces (for example, network and security appliances).

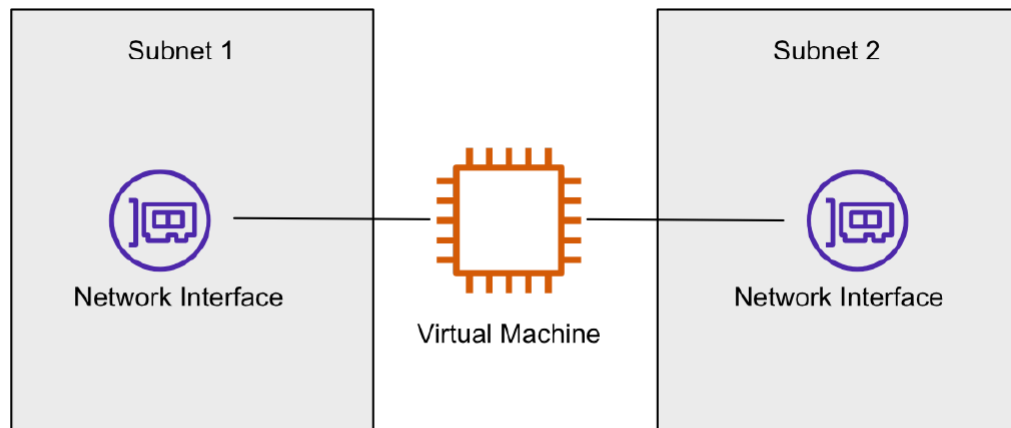


Figure 3.27 A virtual machine with two network interfaces in two different subnets

In the following, you will use an additional network interface to connect a second public IP address to your EC2 instance. Follow these steps to create an additional networking interface for your virtual machine:

1. Open the Management Console, and go to the EC2 service.
2. Select Network Interfaces from the submenu.
3. The default network interface of your virtual machine is shown in the list. Note the subnet ID of this network interface.
4. Click Create network interface.
5. Enter 2nd interface as the description as shown in figure Figure 3.28.
6. Choose the subnet you noted down in step 3.
7. Select Auto-assign private IPv4 address.
8. Chose the security group named launch-wizard-1.
9. Click Create network interface.

After the new network interface's state changes to Available, you can attach it to your virtual machine. Select the new 2nd interface network interface, and choose Attach from the Actions menu. A dialog opens like shown in figure. Choose the only available Instance ID, and click Attach.

Create network interface

An elastic network interface is a logical networking component in a VPC that represents a virtual network card.

Details [Info](#)

Description - optional
A descriptive name for the network interface.

2nd interface

Subnet
The subnet in which to create the network interface.

subnet-2dbf1d5b

Private IPv4 address
The private IPv4 address to assign to the network interface.

☒ Auto-assign
☐ Custom

Elastic Fabric Adapter

☐ Enable

► Advanced settings

Security groups (1/2) [Info](#)

Find resources

	Group ID	Group name	Description
☒	sg-0cb96b57357f61671	launch-wizard-1	launch-wizard created 2022-04-07T...
☐	sg-f25bbe95	default	default VPC security group

Figure 3.28 Creating an additional networking interface for your virtual machine

Attach network interface

Network interface
eni-0865886f80fcc31a9

Instance
i-0c5b91f6d61f2d187

Cancel **Attach**

Figure 3.29 Attaching an additional networking interface to your virtual machine

You've attached an additional networking interface to your virtual machine. Next, you'll associate an additional public IP address to the additional networking interface. To do so, note down the network interface ID of the 2nd interface shown in the overview— `eni-0865886f80fcc31a9` in our example— and follow these steps:

1. Open the AWS Management Console, and go to the EC2 service.
2. Choose Elastic IPs from the submenu.
3. Click Allocate Elastic IP address as you did in section 3.6.
4. Select the newly created public IP address, and choose Associate Elastic IP address from
5. the Actions menu.
6. Select Network interface as the resource type as shown in figure Figure 3.30.
7. Select your 2nd interface's ID.
8. Select the only available private IP of your network interface.
9. Click Associate to finish the process.

The screenshot shows the 'Associate Elastic IP address' page in the AWS Management Console. The page title is 'Associate Elastic IP address' and the subtitle is 'Choose the instance or network interface to associate to this Elastic IP address (3.105.248.200)'. The 'Elastic IP address: 3.105.248.200' is displayed at the top. Under 'Resource type', the 'Network interface' radio button is selected. A callout box points to this selection with the text 'Chose Network interface as resource type.' Below this, a warning message states: 'If you associate an Elastic IP address with a network interface, any previously associated Elastic IP address will be disassociated but still allocated to your account. [Learn more](#)'. The 'Network interface' dropdown menu is set to 'eni-0865886f80fcc31a9', with a callout box saying 'Chose the network interface created in the previous step.' The 'Private IP address' dropdown menu is set to '172.31.47.158', with a callout box stating 'Only a single private IP address available.' At the bottom right, the 'Associate' button is highlighted, with a callout box saying 'Associate the public IP address with the ENI.' The 'Reassociation' section at the bottom has a checkbox 'Allow this Elastic IP address to be reassociated' which is unchecked. 'Cancel' and 'Associate' buttons are at the bottom right.

Figure 3.30 Associating a public IP address with the additional networking interface

Your virtual machine is now reachable under two different public IP addresses. This enables you to serve two different websites, depending on the public IP address. You need to configure the web server to answer requests depending on the public IP address.

Use the Session Manager to connect to your EC2 instance named sydney and execute `ifconfig` in the terminal which will output the network configuration of your virtual machine.

```
$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 9001
    inet 172.31.33.219 netmask 255.255.240.0 broadcast 172.31.47.255 ❶
    inet6 fe80::495:5fff:fea6:abde prefixlen 64 scopeid 0x20<link>
    ether 06:95:5f:a6:ab:de txqueuelen 1000 (Ethernet)
    RX packets 68382 bytes 80442006 (76.7 MiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 35228 bytes 4219870 (4.0 MiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 9001
    inet 172.31.47.158 netmask 255.255.240.0 broadcast 172.31.47.255 ❷
    inet6 fe80::4a2:8fff:feea:bbba prefixlen 64 scopeid 0x20<link>
    ether 06:a2:8f:ea:bb:ba txqueuelen 1000 (Ethernet)
    RX packets 22 bytes 1641 (1.6 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 33 bytes 2971 (2.9 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

[...]
```

- The primary network interface uses private IP address 172.31.33.219.
- The secondary network interface uses private IP address 172.31.47.158.

Each network interface is connected to a private and a public IP address. You'll need to configure the web server to deliver different websites depending on the IP address. Your virtual machine doesn't know anything about its public IP address, but you can distinguish the requests based on the private IP addresses.

First you need two websites. Run the following commands on your virtual machine in Sydney via Session Manager to download two simple placeholder websites:

```
$ sudo -s
```

```
$ mkdir /var/www/html/a
```

```
$ wget -P /var/www/html/a \
```

```
[CA] https://raw.githubusercontent.com/AWSInAction/code3/main/chapter03/a/index.html
```

```
$ mkdir /var/www/html/b
```

```
$ wget -P /var/www/html/b \
```

```
[CA] https://raw.githubusercontent.com/AWSInAction/code3/main/chapter03/b/index.html
```


Next you need to configure the web server to deliver the websites depending on which IP address is called. To do so, add a file named `a.conf` under `/etc/httpd/conf.d`. For example, by using the editor `nano`.

```
nano /etc/httpd/conf.d/a.conf
```

Copy and paste the following file content. Change the IP address from `172.31.x.x` to the IP address from the `ifconfig` output for the networking interface `eth0`:

```
<VirtualHost 172.31.x.x:80>
```

```
DocumentRoot /var/www/html/a
```

```
</VirtualHost>
```

Press `CTRL + X` and select `y` to save the file.

Repeat the same process for a configuration file named `b.conf` under `/etc/httpd/conf.d` with the following content. Change the IP address from `172.31.y.y` to the IP address from the `ifconfig` output for the networking interface `eth1`:

```
<VirtualHost 172.31.y.y:80>
```

```
DocumentRoot /var/www/html/b
```

```
</VirtualHost>
```

To activate the new web server configuration, execute `systemctl restart httpd`.

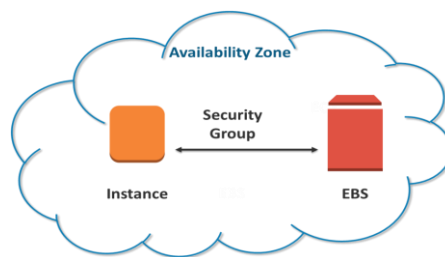
Next, go to the Elastic IP overview in the Management Console. Copy both public IP addresses, and open them with your web browser. You should get the answer “Hello A!” or “Hello B!” depending on the public IP address you’re calling. Thus you can deliver two different websites, depending on which public IP address the user is calling. Congrats— you’re finished!

Rapid Elasticity using Amazon EBS, Amazon EFS, Amazon VPC

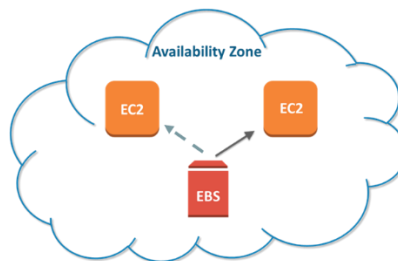
Amazon EBS

Elastic Block Store (EBS) is a block storage service provided by Amazon Web Services (AWS) for use with Amazon Elastic Compute Cloud (EC2) instances. It offers durable, high-performance block-level storage volumes that can be attached to EC2 instances, providing persistent storage for data.

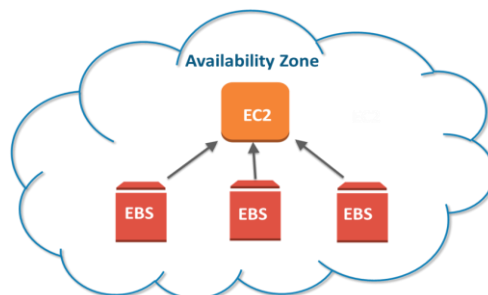
- EBS is a logical volume associated with the EC2 instances
- This type of storage is used when the data needs to be accessed quickly and is required for a long time
- Lifetime of the EBS is not dependent on the EC2 instance
- EBS volume and instance must be located in the same Availability Zone



- It can be detached and attached between the instances in the same Availability Zone



- Any number of EBS Volumes can be attached with an EC2 instance



Features of EBS:

Persistent Storage: EBS volumes persist independently from the life of an EC2 instance. This means that even if you stop or terminate an EC2 instance, the data stored on its attached EBS volumes remains intact.

High Performance: EBS volumes provide consistent and low-latency performance, making them suitable for a wide range of applications, from small databases to high-performance computing workloads.

Scalability: EBS volumes can be easily resized to meet changing storage requirements without any downtime. You can increase or decrease the size of an EBS volume and adjust its performance characteristics as needed.

Data Encryption: EBS volumes can be encrypted using AWS Key Management Service (KMS) keys to protect data at rest. This helps ensure the confidentiality and integrity of data stored on EBS volumes.

Snapshot Backups: EBS volumes can be backed up using snapshots, which are point-in-time copies of the volume's data stored in Amazon Simple Storage Service (S3). Snapshots are incremental, meaning that only the changes made since the last snapshot are stored, reducing storage costs and improving backup efficiency.

High Availability and Durability: EBS volumes are designed for high availability and durability. They are replicated within a single Availability Zone (AZ) to protect against component failures. Additionally, you can create snapshots and copy them across multiple AZs or regions for added redundancy.

Types of EBS Volumes:

General Purpose (SSD): Also known as gp2 volumes, these provide a balance of price and performance for a wide variety of workloads. They offer low-latency performance and are suitable for most applications.

Provisioned IOPS (SSD): Also known as io1 volumes, these are designed for I/O-intensive workloads that require consistent and predictable performance. You can specify the desired number of IOPS (Input/Output Operations Per Second) when creating io1 volumes.

Throughput Optimized (HDD): Also known as st1 volumes, these are designed for frequently accessed, throughput-intensive workloads, such as big data processing and data warehousing.

Cold HDD: Also known as `sc1` volumes, these provide low-cost storage for infrequently accessed data. They are suitable for scenarios where cost optimization is a priority and performance requirements are moderate.

Magnetic (Standard): Also known as magnetic volumes, these offer the lowest cost per gigabyte of all EBS volume types. They are suitable for workloads with low-performance requirements or where cost is the primary consideration.

Rapid elasticity means that you can quickly scale up or down the amount of storage space you're using. For example, if you suddenly need more storage space because your application is growing, you can easily increase the capacity. Conversely, if you find that you're using more storage space than necessary, you can reduce it just as easily.

Elastic Block Storage (EBS) is like having a virtual hard drive for your computer in the cloud.

Imagine you have a computer where you can store all your files and data.

In the cloud, your computer is virtual, but you still need a place to store your stuff. That's where EBS comes in.

The "elastic" part of EBS means that you can easily adjust the size of your storage volumes as needed. If you need more space, you can simply increase the size of your EBS volume. And if you no longer need as much space, you can decrease it. This flexibility makes it easy to scale your storage up or down depending on your needs.

First we understand the terminology of vertical and horizontal scaling, and then we'll dive into Amazon Elastic Block Store (EBS) in AWS.

1. Vertical Scaling:

Vertical scaling, also known as scaling up or scaling vertically, involves increasing the capacity of a single server or instance by adding more resources such as CPU, memory, or storage. In this approach, you typically upgrade the existing hardware components of the server to handle increased workload or resource requirements. For example, you might add more RAM to a server to improve its performance.

Vertical scaling has its limitations, as there's only so much you can upgrade a single server before reaching its maximum capacity. It also introduces potential points of failure, as all resources are concentrated in a single instance.

2. Horizontal Scaling:

Horizontal scaling, also known as scaling out or scaling horizontally, involves adding more servers or instances to distribute the workload across multiple machines. Instead of upgrading a single server, you add more servers to your infrastructure to handle increased demand. Each server operates independently and can share the workload with other instances.

Horizontal scaling offers greater scalability compared to vertical scaling, as it allows you to add more resources as needed without being limited by the capacity of a single server. It also provides redundancy and fault tolerance, as multiple instances can continue to operate even if one or more servers fail.

Key features and benefits of Amazon EBS:

1. Elasticity: With Amazon EBS, you can easily scale your storage capacity up or down based on your application's needs. You can create EBS volumes of different sizes and types (such as General Purpose SSD, Provisioned IOPS SSD, and Throughput Optimized HDD) and attach them to your EC2 instances as needed.

2. Durability and Availability: Data stored on EBS volumes is replicated within a single Availability Zone (AZ) to protect against component failure. You can also create snapshots of your EBS volumes to back up your data and restore it in case of data loss or corruption.

3. Performance: Amazon EBS offers different types of storage volumes optimized for different performance requirements. For example, General Purpose SSD volumes provide a balance of price and performance for a wide range of workloads, while Provisioned IOPS SSD volumes offer predictable performance for I/O-intensive applications.

4. Snapshots and Backup: You can create point-in-time snapshots of your EBS volumes, which are stored in Amazon S3. These snapshots can be used to back up your data, migrate volumes between Availability Zones, or create new volumes.

Create and configure storage services using Amazon EBS

Amazon EBS (Elastic Block Store): Amazon EBS is a high-performance block storage service designed for use with EC2 instances. It provides persistent storage for virtual machines, meaning data remains available even after the instance is stopped or terminated.

EBS volumes can be attached to EC2 instances and support features like snapshots, encryption, and scalability. Scale Up and Scale Down.

Scale Up /Scale Down (Vertical Scaling):

Definition: Scale up, also known as vertical scaling, involves increasing the resources (such as CPU, RAM, or disk space) of an existing single server or virtual machine to handle increased workload or performance requirements.

EBS Equivalent: increasing the size or performance specifications of an existing EBS volume

Procedure: increasing/decreasing of CPU and RAM

Step 1. - Navigate to the EC2 dashboard.

Step 2. - Stop the EC2 instance associated with the EBS volume.

Step 3. - Click "Actions" and then—instance settings—change the instance type"Modify Volume".

Step 4. -Increase/ decrease the size or change the volume type to a higher performance specification.

Step 5. - Click "Modify" to apply the changes.

Step 6: Now the start the machine you see with increase /decrease of size and performance

Attaching and Detaching Volumes with in same region of EC2

Step 1. Launching an Instance with Additional Volumes:

- Start by launching an instance from the EC2 dashboard. Choose your desired AMI (Amazon Machine Image), instance type, and other configurations.

- In the "Add Storage" step, you can add additional volumes. Specify the size and other attributes for these volumes.

Step 2. Detaching a Volume from an Instance:

- Navigate to the EC2 and select "Volumes" under EBS section from the left-hand menu.
- Locate the volume you want to detach from the instance.
- Select the volume, then choose "Actions" -> "Detach Volume".
- Confirm the detachment by clicking "Yes, Detach".

Step 3. Attaching a Volume to Another Instance:

- After detaching the volume, it will become available.
- Select the detached volume from the Volumes list.
 - Choose "Actions" -> "Attach Volume".
 - Select the instance you want to attach the volume to
- specify the device name (e.g., /dev/sdf), and then click "Attach".
 - Once attached, the volume will appear as an additional disk on the chosen instance.

Step 4: Attach and Mount EBS Volume to Linux EC2

Attach the volume via AWS Console.

- Connect to EC2 via SSH.
- Check available disks (lsblk).

Here you see root volume and attached (eg:-xvdf)

```
lsblk -fs # Check filesystem type
```

Create a partition using fdisk:

```
fdisk /dev/xvdf
```

- o Press m → Show help menu
- o Press n → Create a new partition
- o Press p → Create a primary partition

- o Press Enter twice → Use default partition size
- o Press w → Write changes and exit

Partprobe # Inform the kernel about partition changes

Verify the new partition:

lsblk -fs

Format the Disk Partition

mkfs.xfs /dev/xvdf1 # For XFS

Verify the filesystem:

lsblk -fs

Mount the Partition

Steps:

Create a directory for mounting:

1. mkdir /mnt/archana

Mount the partition:

2. mount /dev/xvdf1 /mnt/archana

Verify the mount:

3. df -h

Make the Mount Persistent

Steps:

Edit /etc/fstab to add the following line:

nano /etc/fstab

Add this line at the end:

<device> <mount_point> <filesystem_type> defaults 0 0

Now you see your volume is persistent

Check it by detaching the volume from instance and attaching it to another instance...

Note :- you need to create the directory and mount it ...to retrieve the files

Attaching Volumes across regions of EC2 using the snapshot

To create a snapshot of an EBS volume and attach it to an instance in another region in AWS, you'll need to follow these steps:

Step 1. Create a Snapshot:

- a. Sign in to the AWS Management Console.
- b. Navigate to the EC2 Dashboard.
- c. Click on "Volumes" in the left-hand navigation pane.
- d. Select the EBS volume you want to create a snapshot of.
- e. Click on the "Actions" dropdown menu above the volume list and select "Create Snapshot."
- f. Provide a name and description for the snapshot.
- g. Click on the "Create Snapshot" button to initiate the snapshot creation process.

Step 2. Copy the Snapshot to Another Region:

- a. Once the snapshot is created, go to the "Snapshots" section in the EC2 Dashboard.
- b. Select the snapshot you just created.
- c. Click on the "Actions" dropdown menu above the snapshot list and select "Copy Snapshot."
- d. Choose the destination region where you want to copy the snapshot.
- e. Click on the "Copy Snapshot" button to initiate the copy process. This may take some time depending on the size of the snapshot and the network speed.

Step 3. Monitor the Snapshot Copy Progress:

- a. You can monitor the progress of the snapshot copy by navigating to the "Snapshots" section in the EC2 Dashboard of the source region.
- b. Look for the snapshot you copied and check its status. It will change from "pending" to "completed" once the copy process is finished.

Step 4. Switch to the Destination Region:

- a. Use the region selector in the top-right corner of the AWS Management Console to switch to the destination region where you copied the snapshot.

Step 5. Create a Volume from the Snapshot:

- a. In the EC2 Dashboard of the destination region, go to the "Snapshots" section.
- b. Find the snapshot you copied from the source region.
- c. Click on the snapshot, then click on the "Actions" dropdown menu and select "Create Volume."
- d. Configure the volume settings, such as volume type, size, and availability zone.
- e. Click on the "Create Volume" button to create the volume from the snapshot.

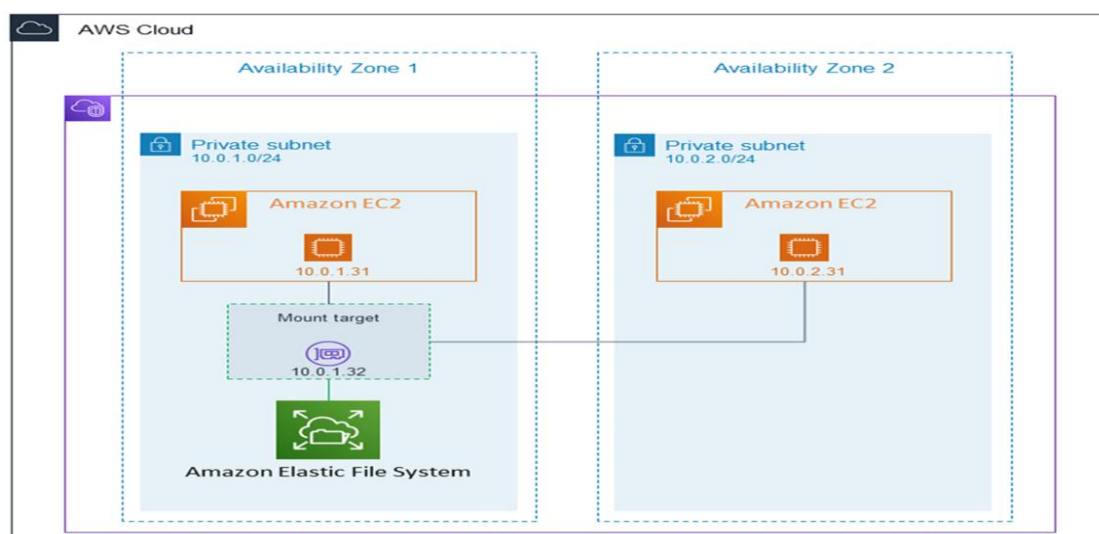
Step 6. Attach the Volume to an Instance:

- a. Once the volume is created, navigate to the "Volumes" section in the EC2 Dashboard.
- b. Find the newly created volume and select it.
- c. Click on the "Actions" dropdown menu and select "Attach Volume."
- d. Choose the EC2 instance to which you want to attach the volume and specify the device name.
- e. Click on the "Attach" button to attach the volume to the instance.

We have created a snapshot of an EBS volume, copied it to another region, created a volume from the snapshot, and attached it to an instance in the destination region.

Elastic File Storage

Amazon Elastic File System (EFS) is a scalable and fully managed file storage service provided by Amazon Web Services (AWS). It allows you to create and configure file systems that can be accessed concurrently from multiple EC2 instances, providing a highly available and scalable storage solution for your applications and workloads.



Amazon EFS supports for only Amazon Linux instance:

Step 1: Launching EC2 Instances

1. Sign in to AWS Management Console.
2. Go to the EC2 dashboard.
3. Click on "Launch Instance".
4. Configure instance details:
 - Instance Name: EFS-
5. Choose an Amazon Machine Image (AMI) for Linux.
6. Select instance type: t2.micro.
 - Key Pair: EFS

- Network: Choose Subnet-1a
- Security Group: Create a new security group (SG) and add NFS and allow from anywhere.

7. Launch the instance.

8. Repeat the above steps to launch another instance named EFS-2 in Subnet-1b with the same configuration.

Step 2: Creating an EFS File System

1. Go to the EFS service in the AWS Management Console.
2. Click on "Create file system".
3. Specify details:

- Name: Optional
- VPC: Default
- Enable region button.

Click on "Next".

4. In the Network settings:

- Delete all security groups.
- Select the newly created security group (NFS).

Click on "Next" and review the configuration.

Click on "Create file system".

Step 3: Accessing the two EC2 instances named EFS-1 & EFS -2 in two different PowerShell sessions and performing the specified tasks:

Accessing EFS-1 Instances in Two Different PowerShell Sessions:

1. Open Two PowerShell Sessions:

- Open two separate PowerShell windows or tabs on your local machine.

For each Instance:

2. SSH into the Instance:

- Use the SSH command to connect to the EFS-1 instance. Replace `[instance-public-ip]` with the public IP address of the instance:

```
ssh -i [path-to-your-keypair.pem] ec2-user@[instance-public-ip]
```

3. Switch to Root User:

- Gain root access by executing the following command:

```
sudo su
```

4. Create a Directory:

- Make a directory named "efs" using the following command:

```
mkdir efs
```

5. Install Amazon EFS Utilities:

- Use yum package manager to install the Amazon EFS utilities:

```
yum install -y amazon-efs-utils
```

6. List Files:

- Execute the following command to list files in the current directory:

```
ls
```

7. Verify Installation (Optional):

- Optionally, you can verify the installation of the Amazon EFS utilities by checking the version:

```
efs-utils --version
```

Repeat Steps 2-7 for Each PowerShell Session Corresponding to EFS-2 Instance.

By following these steps, you'll have accessed each EFS-1 instance in separate PowerShell sessions, switched to the root user, created a directory named "efs," installed the Amazon EFS utilities, and listed files in the directory. This setup allows you to configure and manage each instance individually as needed.

Step 4: Attaching EFS to EC2 Instances

1. Go to the EFS service in the AWS Management Console.
2. Click on the target EFS file system.
3. Click on the "Attach" button.
4. Choose "Mount via DNS" option.
5. Copy the displayed command.
6. SSH into each EC2 instance.
7. Paste and execute the copied command in the terminal to mount the EFS file system onto the instance.

Step 5: Verify and Test EFS

1. Change the directory to EFS on both instances.
2. Create a file on one instance.
3. Verify that the file automatically synchronizes and appears on the other instance.

By above steps, we can successfully create EC2 instances, configured them, created an EFS file system, and attached it to the instances in the same availability zone in the Mumbai region. Now, the instances can seamlessly access and share files stored in the EFS file system.

Virtual Private Network (VPC)

A Virtual Private Cloud (VPC) is a virtual network environment in the Amazon Web Services (AWS) cloud that closely resembles a traditional network that you might operate in your own data center.

It allows you to provision a logically isolated section of the AWS cloud where you can launch AWS resources in a virtual network that you define.

VPCs give you control over your network configuration, including IP address ranges, subnets, routing tables, network gateways, and security settings

CIDR Block:

CIDR is a method used to represent IP addresses and specify address ranges in a more flexible and efficient way than traditional IP address classes (Class A, B, and C).

- With CIDR, an IP address is followed by a slash ("/") and a number, known as the prefix length or subnet mask.

- The prefix length indicates the number of bits used for the network portion of the address.

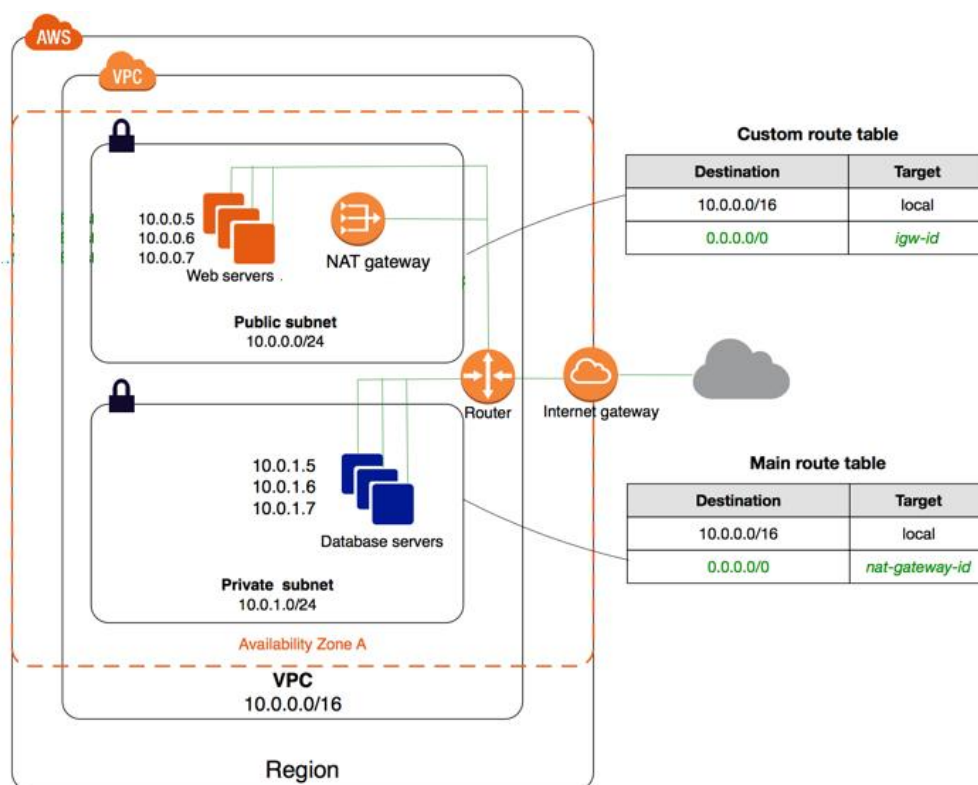
- For example, in the CIDR notation "192.168.1.0/24", the "/24" indicates that the first 24 bits represent the network portion of the address, leaving 8 bits for the host portion.

- For example, you might define a CIDR block of 10.0.0.0/16, which allows for up to 65,536 IP addresses.

Subnets:

- Subnets are segments of the VPC's IP address range where you can place groups of resources, such as EC2 instances, RDS databases, and Lambda functions.

- Each subnet resides in a specific Availability Zone (AZ) and has its own CIDR block, which is a subset of the VPC's CIDR block.



Steps to create a VPC (Virtual Private Cloud) network in AWS with one public server and another private server:

Step 1: Create a VPC

1. Sign in to the AWS Management Console.
2. Go to the VPC dashboard.
3. Click on "Create VPC."
4. Enter the following details:
 - Name tag: Provide a name for your VPC (e.g., MyVPC).
 - IPv4 CIDR block**: Define the IP address range for your VPC (e.g., 10.0.0.0/16).
5. Click on "Create."

Step 2: Create Subnets

1. In the VPC dashboard, click on "Subnets" in the left-hand navigation pane.
2. Click on "Create subnet."
3. Enter the following details:
 - Name tag: Provide a name for your subnet (e.g., PublicSubnet).
 - VPC: Select the VPC created in Step 1.
 - Availability Zone: Choose an availability zone.
 - IPv4 CIDR block: Define the IP address range for the subnet (e.g., 10.0.1.0/24 for a public subnet).
4. Click on "Create."

Note: Every subnet will be by default private.

We want to make one subnet as public.

To make subnet public its two step process.

Step 1: we need to enable public IP

Select the subnet (10.0.1.0/24) --> Actions --> Modify Auto Assigning IP Settings

---> Enable Auto Assigning public IPv4 Address -- Save

(From now, public IP will be assigned to the machines in this Subnet)

Repeat the above steps to create another subnet for the private server (e.g., PrivateSubnet with an IP address range like 10.0.2.0/24).

Step 3: Create Internet Gateway (IGW)

1. In the VPC dashboard, click on "Internet Gateways" in the left-hand navigation pane.
2. Click on "Create internet gateway."
3. Attach the internet gateway to your VPC.

Step 4: Update Route Table for Public Subnet

1. In the VPC dashboard, click on "Route Tables" in the left-hand navigation pane.

Now, we need to connect Route table to Subnet.

Select the route table (InternetRT) ---> Subnet Associations tab ---> Edit Subnet Associations
---> Select the subnet (10.0.1.0/24) -- Save

2. Select the route table associated with the public subnet.
3. Click on the "Routes" tab and then "Edit routes."
4. Add a route with destination `0.0.0.0/0` and target the internet gateway created in Step 3.
5. Click on "Save routes."

Step 5: Launch Instances

1. Go to the EC2 dashboard.
2. Launch a new EC2 instance for the public server:
 - Select the appropriate AMI (Amazon Machine Image).
 - Choose the public subnet created in Step 2.
 - Configure security groups to allow inbound traffic on port 80 (HTTP) or other necessary ports.
3. Launch another EC2 instance for the private server:

- Select the appropriate AMI.
- Choose the private subnet created in Step 2.
- Ensure the security group only allows necessary inbound traffic from the public server or other trusted sources.

Step 6: Access Configuration

- The public server will have a public IP address and can be accessed directly over the internet.
- The private server will not have a public IP address and can only be accessed from within the VPC or through a VPN connection.